

📖 OOAD Ch8 課堂筆記：Middle Level Notations、ERD 與正規化

主題重點：為何 [Object-Oriented Analysis and Design](#) 要談資料庫設計、[ERD](#) 的起源、[UML](#) 標記法、多對多關聯的正規化，以及校務系統的類別分析思路。

📖 課程引言：為何需要 Middle Level Notations?

- 雖然這門課是 物件導向分析與設計 (OOAD)，但老師強調：資料庫正規化與物件類別分析在本質上非常接近。
- 最終我們會得到具體的資料表 (tables) 或類別 (classes)，但如果只看到最後成果，別人很難逆推出當初的分析邏輯。
- 一旦系統進入維護階段，如果中間分析過程遺失，就會很難準確修改。
- 因此 1970 年代研究者開始尋找一套可重複操作的方法論 (methodology)，希望像 SOP 一樣，讓分析者依循步驟就能做出高正確率的設計。
- Middle Level Notations 的價值就在於：
 - 保留分析過程
 - 視覺化設計思路
 - 協助他人檢查設計是否合理

🖼️ Data-Oriented Models 與 ERD 的起源

- 為了替 [database design](#) 建立標準樣板，學界曾提出許多不同的表示法。
- 最後由 Peter Chen 在 1976 年提出 [ERD](#)，成為當時最重要的資料分析與系統分析工具。

ERD 的兩大貢獻

- The Entity-Relationship Principle
 - 強調先思考：我們到底要保存哪些資料？
- The Entity-Relationship Notation
 - 提供一套圖形化的表示法，讓分析結果可視化。

老師強調的資料觀點

- 資料是企業資產 (Corporate Asset)
- 資料比程序更穩定 (Stability of Data)

也就是說，企業的流程可能會改、系統功能可能會換，但只要業務本質不變，真正要處理的核心事物通常不會劇烈改變。

🌱 The Entity-Relationship Principle：先看「東西」，不是先看程序

早期分析方法偏向程序導向，重點是「使用者要做什麼」。

ER 原則則更進一步地問：使用者必須知道哪些東西 (things) 才能完成工作？

1. Entity (實體)

- 實體是指：在使用者世界中，具有 獨立且可區分存在性 的事物。
- 只有當我們對這個事物感興趣，且需要保留它的資料來支援業務運作時，它才適合成為 entity。

例如：

- 電話公司會關心 [Customer](#)、[Phone Number](#)
- 售票系統會關心 [Venue](#)、[Artist](#)、[Seat](#)

2. 為什麼 Date 不是 Entity?

- [Date](#) 本身不能代表一個具有獨立個體的事物。
- 單看 [2012/12/25](#)，你無法知道它對應的是哪一個具體事件。
- 所以 [Date](#) 比較適合作為 屬性 (attribute)，而不是 實體 (entity)。

3. Entity Type vs. Occurrence / Instance

- Entity Type：某一類實體的型別或分類
 - 在資料庫中可對應為 **table name**
 - 在物件導向中可對應為 **class**
- Occurrence / Instance：某一個具體出現的個體
 - 在 OO 中就是某個 class 的 **instance / object**

例子：

- [Student](#) 是 entity type
- 某一位特定學生是 [Student](#) 的 instance

4. Attributes (屬性)

- 屬性是描述 entity 的資料元素。
- 它們用來記錄 entity 的狀態與特徵。

5. Associations (關聯)

- 光有很多 customer 和 product，並不代表有商業行為。
- 當 customer **buys** product，才形成了真正有意義的互動。
- 關聯通常用 動詞 表示，並提供資料之間的 存取路徑 (access paths)。

老師的設計提醒

- 好的物件導向設計，應該符合人類世界的 常識 (common sense)。
- 如果只專注在語法或技術炫技，常會設計出很怪、很難維護的 class 結構。

🕒 物件導向真正的歷史脈絡：從程序導向失敗到資料中心思維

這一段是老師認為最關鍵的背景。

1. 程序導向的困境

- 早期軟體工程與程式語言如 **BASIC**、**Pascal**、**Ada**、**Modula-2** 等，主要聚焦在 **procedure-oriented** 的設計。
- 這些方法雖然強調結構化，但在面對大型系統維護時仍然失敗。
- 常見問題就是：改一小部分，其他地方也跟著壞掉。

2. 資料庫領域的發現

- 1970 年代資料庫學者早就發現：真正穩定的是資料，不是功能。
- 老師提到一個資料模型師的信條：

Data is the Centre of the Universe.

例如：

- 客運公司的系統功能可能 10 年後改變
- 但它要管理的仍然是 **Bus**、**Driver**、**Route**、**Passenger**

3. Smalltalk 的誕生

- **Xerox PARC** 的研究團隊在面對程序導向維護困境後，重新回頭閱讀資料庫與資料模型相關文獻。
- 他們發現：「功能常變，資料較穩定。」
- 因此系統核心從 **procedure** 轉向 **data**，並逐步發展出 **object-oriented** 的思維，最後誕生了 **Smalltalk**。

4. 物件導向的真正價值

老師特別提醒：

- 物件導向的精華 不在於 繼承、多型這些看起來很炫的語法特性。
- 真正的核心是：

先把穩定的資料結構設計對。

- 當資料結構合理時，未來功能擴充與維護時，需要修改的程式碼就會少很多。

ERD 與 UML 標記法

雖然過去 ERD 曾經有很多不同畫法（例如橢圓形表示屬性、菱形表示關聯），但現在課堂要求統一使用 **UML** 來表達。

1. Entity / Class 的畫法

- 用方框表示
- 上方寫 **單數名詞**（例如 **Student**）
- 下方列出屬性
- 主鍵（Primary Key）放第一個，並用 * 標示

範例概念：

```
Student
-----
* studentId
  nationalId
  name
```

2. Associations 的畫法

- 用一條線連接兩個實體
- 上方標註動詞
- 可以加箭頭來表示句子的閱讀方向

例如：

- **Student enrolls in Course**
- **Instructor teaches Course Offering**

3. Multiplicity / Cardinality

- 用來表示參與關聯的實例數量
- **1** 表示一個
- * 表示多個 (many)

一對多 (1:M)

例如：

- 一位老師教很多課
- 一門課通常對應一位老師

實作上可能會出現：

- **Instructor** 內有一個 **vector<Course*>**
- **Course** 內有一個指向 **Instructor** 的參考

多對多 (M:M)

例如：

- 很多 customer 會買很多 product
- 很多 student 會修很多 course

多對多關聯與正規化

老師強調：圖上出現多對多 (M:M) 時，通常代表正規化還不夠。

為什麼 M:M 關聯麻煩？

假設直接這樣設計：

- `Student` 有 `vector<Course*>`
- `Course` 有 `vector<Student*>`

一開始看似合理，但如果主管突然要求：

- 記錄「某個學生修某門課的到課時數」
- 或記錄該次選課的成績、狀態、修課學期

這些資料：

- 不屬於 `student` 本身
- 也不屬於 `course` 本身
- 而是屬於 **`student`** 與 **`course`** 的那一段關係

這時如果沒有正規化，就會不知道該把欄位放在哪裡，系統也會變得難維護。

正規化解法：建立中介類別 / 關聯表

把原本的 M:M 直接拆掉，在中間新增一個類別：

- `Enrollment`
- `SupplyRelation`
- `Registration`

這個中介類別：

- 以兩端的主鍵作為 foreign keys
- 與左右兩側分別形成兩個 1:M 關係
- 可以自己攜帶額外屬性

供應商與零件的例子

老師用 `Supplier` 與 `Part` 說明：

- 一個 `supplier` 可以供應很多 `parts`
- 一個 `part` 也可以由很多 `suppliers` 供應

這是典型 M:M，因此需要建立中介關係，例如 `SupplyRelation`。

`SupplyRelation` 可以帶有：

- `supplierId`
- `partId`
- `qty`
- `city2`（零件儲存城市）

這樣一來就能清楚表示：

- 哪個供應商供應哪種零件
- 數量是多少
- 存在哪個城市

也讓未來增加更多關聯屬性時仍能維持清楚的資料結構。

課堂練習：校務行政系統 UML 分析

老師給出的需求包括：

- 一位老師可以教很多課
- 學生可以自由選修課程
- 課程有時數、教室、老師等資訊
- 學生有身分證字號、學號等資訊
- 學生會被編入某個班級
- 每個班級有一位老師擔任導師

可辨識出的主要類別

- `Student`
- `Teacher`
- `Course`
- `Classroom` 或 `Class`
- `Enrollment`（若學生與課程之間是多對多）
- `Subject`（視需求而定）

Subject 要不要獨立成 Table？

老師提出了一個很重要的分析判斷：

- 如果 `Subject` 只是像「英文、數學」這種幾乎不變的名稱，且沒有其他屬性需求，可能只需要用一個字串欄位即可。
- 但如果你未來需要：
 - 課程描述
 - 指定教科書
 - 學分資訊
 - 科目分類

那麼把 `Subject` 獨立成一個 entity / table 會是比較好的設計。

這裡的關鍵判準

不是「能不能拆」，而是：

這個概念是否有足夠獨立性，值得單獨保存資料？

如果答案是 yes，它就有機會成為 entity。

✔ 本章核心觀念總結

1. **OOAD** 與資料庫設計密切相關，尤其是在類別與關聯分析上。
2. **Middle Level Notations** 的價值在於保留分析脈絡，而不是只留下最終資料表。
3. **ERD** 的核心是先找 **entity**、**attribute**、**association**。
4. 資料比功能穩定，這也是物件導向能成立的重要歷史背景。
5. **UML** 是目前課堂要求的標準表示法。
6. 多對多關聯通常要正規化，改成透過中介 entity 連接的兩個 1:M。
7. 是否將某概念獨立成 class / table，取決於它是否具有 **獨立存在性** 與 **額外屬性需求**。

🔗 Related Notes

-  [School/OOAD_Object-Oriented Analysis and Design](#)
- [School/OOAD/QA5_OOAD_Class_Notes](#)
- [School/OOAD/CH7_HW_Answer](#)